

صورت پروژه درس مبانی کامپیوتر و برنامه سازی

دکتر امین طوسی - ترم پاییز ۱۴۰۴

این پروژه شامل پیاده سازی یک دستگاه Mini Arcade دیجیتال است که شما می توانید از میان لیست بازی ها، چند بازی را برای پیاده سازی انتخاب کنید. هر بازی یک «قانون تغییر یافته» دارد و طبق ضریب مشخص شده در ادامه ارزش گذاری شده است. هدف این است که شما در نهایت به نمره ضریب Core مساوی با ۱.۰۰ برسید. اگر نمره بخش Core بیشتر از ۱ شود، همچنان نمره از ۱ محاسبه می شود.

همچنین هر بازی بخشی نمره مازاد دارد که مجموع نمره همه بخش های مازاد روی هم رفته حداکثر ۰.۲۵ است و مستقل از نمره بخش Core محاسبه می شود. موارد امتیاز مازاد هر بازی در پایان توضیحات بخش Core هر بازی مشخص شده است.

ضرایب Core

ضرایب نمره هر بازی مطابق سختی نسبی پیاده سازی آن است. شما باید ترکیبی از بازی ها را انتخاب کنید که حداقل به ضریب ۱ برسد. اگر Chess را انتخاب کنید مستقیم نمره کامل Core را می گیرید و می توانید باقی بازی ها را پیاده سازی نکنید.

نام بازی	ضریب Core
شطرنج (با مهره های متفاوت)	۱.۰۰
Checkers	۰.۵۵
2048 Fibonacci	۰.۳۵
Hangman Battle	۰.۲۵
TicTacToe - دوز بازی	۰.۲۰
Rock Paper Scissors 5 Move - پنج کاغذ قیچی	۰.۱۵
Number Guess + Hints - حدس عدد با راهنمایی	۰.۱۰

نمره دهی

- نمره Core بر اساس مجموع ضرایب بازی های انتخاب شده محاسبه می شود.
- اگر مجموع بین ۱.۰ تا ۱.۲ باشد، نمره نهایی Core همان ۱.۰ خواهد بود.

- بخش مازاد تا سقف ۰.۲۵ است و جداگانه محاسبه می‌شود.
- نمره نهایی = Core (تا سقف ۱.۰۰) + Extra (تا سقف ۰.۲۵)

منوها و رابط کاربری

در ابتدای اجرای برنامه، باید یک منوی اصلی برای انتخاب بازی در اختیار کاربر قرار گیرد. هر بازی که دارای حالت‌های مختلف است (مانند بازی تک‌نفره یا دونفره)، باید منوی داخلی مخصوص به خود برای انتخاب حالت داشته باشد.

پس از پایان هر بازی، برنامه باید به بازیکن این امکان را بدهد که یا همان بازی را دوباره اجرا کند (Play Again) منوی اصلی بازگردد و بازی دیگری را انتخاب نماید. تمام منوها باید از نظر بصری یکنواخت، هماهنگ و دارای سبک نمایش یکسان باشند.

نمره مازاد رابط گرافیکی

پیاده‌سازی رابط گرافیکی (GUI) برای منوها و بازی‌ها اختیاری است و به‌عنوان نمره مازاد در نظر گرفته می‌شود. الزامی نیست که برای تمام بازی‌ها رابط گرافیکی پیاده‌سازی شود و ارزیابی مازاد صرفاً بر اساس کیفیت و یکپارچگی رابط کاربری انجام خواهد شد. مجموع نمره مازاد رابط گرافیکی نیز در سقف ۰/۲۵ کل پروژه محاسبه می‌شود.

شطرنج تغییر یافته - Core = ۱.۰۰

در این پروژه شما یک نسخه تغییر یافته از بازی کلاسیک شطرنج (Chess) را پیاده‌سازی می‌کنید. صفحه بازی همان صفحه استاندارد ۸×۸ است و دو بازیکن به‌صورت نوبتی بازی می‌کنند. قوانین پایه شطرنج از جمله نوبت‌بندی بازیکنان، گرفتن مهره‌های حریف و مفهوم محافظت از شاه برقرار است.

در این نسخه، قانون کستلینگ (Castling) به‌طور کامل حذف شده است و نیازی نیست پیاده‌سازی شود. در مقابل، ارتقای پیاده Pawn جزو بخش امتیازی محسوب می‌شود و در پیاده‌سازی بازی پایه الزامی نیست.

مفهوم کیش و کیش‌ومات باید به‌صورت دقیق پیاده‌سازی شود. کیش به وضعیتی گفته می‌شود که شاه یک بازیکن در معرض تهدید مستقیم مهره‌های حریف قرار دارد. در این حالت، بازیکن موظف است حرکتی انجام دهد که این تهدید را برطرف کند. اگر بازیکن هیچ حرکت قانونی‌ای نداشته باشد که شاه را از وضعیت کیش خارج کند، وضعیت کیش‌ومات (Checkmate) رخ داده و بازی به پایان می‌رسد.

حرکت مهره‌ها در این نسخه با شطرنج کلاسیک متفاوت است. مهره‌ها دارای قوانین حرکتی ویژه‌ای هستند که باید دقیقاً مطابق تعریف پیاده‌سازی شوند و هیچ حرکت کلاسیکی جایگزین آن‌ها نشود.

شکل ساده صفحه

g	d	t	k	q	t	d	g		8
p	p	p	p	p	p	p	p		7
.	.	.	.	.	.	.	.		6
.	.	.	.	.	.	.	.		5
.	.	.	.	.	.	.	.		4
.	.	.	.	.	.	.	.		3
P	P	P	P	P	P	P	P		2
G	D	T	K	Q	T	D	G		1
-	-	-	-	-	-	-	-		
h	g	f	e	d	c	b	a		

- مهره Gryphon: یک خانه قطری سپس حرکت رخ.
- مهره Dragon: حرکت اسب یا یک قدم در همه جهت‌ها.
- مهره Thief: حرکت فیل با یک پرش از روی یک قطعه.
- مهره Queen: وزیر معمولی + حرکت اسب.
- مهره King: یک یا دو خانه در هر جهت، بدون پرش.

وظیفه شما

شما باید منطق کامل حرکت مهره‌ها را مطابق قوانین جدید پیاده‌سازی کنید و از انجام هرگونه حرکت غیرمجاز جلوگیری نمایید.

کنترل نوبت بازیکنان، تشخیص وضعیت کیش، جلوگیری از باقی‌ماندن شاه در وضعیت کیش، و تشخیص صحیح پایان بازی در حالت کیش‌ومات از وظایف اصلی شماست. همچنین باید ورودی‌های بازیکن را اعتبارسنجی کنید و در صورت وارد شدن حرکت نامعتبر، پیام مناسب نمایش دهید.

نمره مازاد

در صورت پیاده‌سازی کامل و صحیح بخش پایه، امکانات زیر به‌عنوان نمره مازاد در نظر گرفته می‌شوند:

- پیاده‌سازی ارتقای پیاده (Pawn Promotion)
 - افزودن تایمر برای هر بازیکن
 - نمایش هایلایت‌شده حرکت‌ها و مسیرهای ممکن مهره انتخاب‌شده
 - تشخیص بن‌بست
- سهم شطرنج از نمره مازاد بیشترین مقدار را در کل پروژه دارد. با این حال، مجموع نمره مازاد کل پروژه حداکثر برابر با ۰/۲۵ خواهد بود.

(۰.۵۵) Checkers

در این بخش شما نسخه‌ای تغییر یافته از بازی چکرز (Checkers) را روی صفحه ۸×۸ پیاده‌سازی می‌کنید. قوانین پایه بازی، شامل نوبت‌بندی، حرکت روی خانه‌های تیره، گرفتن مهره‌ها از طریق پرش و حذف مهره حریف، برقرار است.

در این نسخه، مهره ساده پس از رسیدن به ردیف آخر می‌تواند به یکی از دو حالت ارتقاء پیدا کند. در صورت انتخاب King، مهره می‌تواند مانند چکرز کلاسیک در هر دو جهت حرکت و پرش انجام دهد. در صورت انتخاب Queen، مهره دیگر قابلیت‌های King را ندارد و تنها ویژگی خاص آن، امکان حرکت عمودی در راستای ستون صفحه است. Queen نمی‌تواند به صورت مورب یا جهشی مانند King حرکت کند مگر در جهت عمودی مجاز.

شکل ساده صفحه بازی

x	.	x	.	x	.	x	.		8
.	x	.	x	.	x	.	x		7
x	.	x	.	x	.	x	.		6
.	.	.	.	.	.	.	.		5
.	.	.	.	.	.	.	.		4
.	o	.	o	.	o	.	o		3
o	.	o	.	o	.	o	.		2
.	o	.	o	.	o	.	o		1
h	g	f	e	d	c	b	a		

وظیفه شما

شما باید منطق حرکت مهره‌ها، پرش اجباری و چندپرش را به‌طور کامل پیاده‌سازی کنید. در صورت وجود امکان پرش، بازیکن نباید اجازه انجام حرکت عادی داشته باشد. پس از هر پرش، باید بررسی شود که آیا امکان پرش مجدد وجود دارد یا خیر.

همچنین لازم است فرآیند ارتقاء مهره در رسیدن به ردیف آخر مدیریت شود و انتخاب نوع ارتقاء از بازیکن دریافت گردد. تشخیص صحیح پایان بازی و اعلام برد نیز جزو وظایف شماست.

نمرهٔ مازاد

نمایش هایلایت‌شدهٔ تمام مسیرهای حرکتی و پرشی ممکن مهره‌ها به‌عنوان نمرهٔ مازاد این بازی در نظر گرفته می‌شود.

۲۰۴۸ Fibonacci (۰.۳۵)

در این بخش شما نسخه‌ای تغییر یافته از بازی ۲۰۴۸ را پیاده‌سازی می‌کنید. بازی روی یک صفحهٔ مربعی انجام می‌شود و بازیکن با استفاده از ورودی‌های جهت‌دار (بالا، پایین، چپ، راست) کاشی‌ها را جابه‌جا می‌کند. در نسخهٔ کلاسیک، هر دو کاشی هم‌ارزش با هم ادغام شده و مقدار آن‌ها دو برابر می‌شود. هدف بازی نیز رسیدن به کاشی با مقدار بالا و ادامهٔ بازی تا زمانی است که حرکت قانونی وجود داشته باشد.

در نسخهٔ فیبوناچی این پروژه، قانون ادغام کاشی‌ها تغییر کرده است. شما باید به‌جای مقادیر توانی ۲، از دنبالهٔ فیبوناچی استفاده کنید. دو کاشی تنها در صورتی با یکدیگر ادغام می‌شوند که مقادیر آن‌ها دو عدد متوالی در دنبالهٔ فیبوناچی باشند. برای مثال، کاشی‌های ۱ و ۲ می‌توانند ادغام شده و مقدار ۳ تولید کنند، یا کاشی‌های ۳ و ۵ می‌توانند به ۸ تبدیل شوند. ادغام کاشی‌هایی که چنین رابطه‌ای ندارند، مجاز نیست.

پس از هر حرکت معتبر، یک کاشی جدید با مقدار ابتدایی مناسب (مانند ۱ یا ۲) باید در یکی از خانه‌های خالی صفحه اضافه شود. بازی زمانی به پایان می‌رسد که هیچ حرکت قانونی برای بازیکن وجود نداشته باشد.

نمونه برد بازی

+	+	+	+	+	+	+	+
	.		.		2		1
+	+	+	+	+	+	+	+
	.		.		.		3
+	+	+	+	+	+	+	+
	.		.		.		.
+	+	+	+	+	+	+	+
	8		.		.		.
+	+	+	+	+	+	+	+

وظیفه شما

در پیاده‌سازی این بازی، شما باید منطق کامل جابه‌جایی کاشی‌ها در هر چهار جهت را پیاده‌سازی کنید و قوانین ادغام مبتنی بر دنباله فیبوناچی را به درستی اعمال نمایید. لازم است اطمینان حاصل کنید که هر کاشی در یک حرکت تنها یک بار ادغام می‌شود و ترتیب ادغام‌ها مطابق قوانین بازی رعایت شده است.

همچنین باید وضعیت پایان بازی را تشخیص دهید و از انجام حرکتهای غیرمجاز جلوگیری کنید. نمایش صفحه، دریافت ورودی از کاربر و به‌روزرسانی وضعیت بازی پس از هر حرکت، از وظایف اصلی شما در این بخش است.

نمره مازاد

در صورت پیاده‌سازی کامل بخش پایه، امکان انتخاب اندازه صفحه بازی توسط کاربر به‌عنوان نمره مازاد در نظر گرفته می‌شود. اندازه صفحه باید مربعی باشد و محدودیتی از نظر مقدار بالایی برای آن در نظر گرفته نمی‌شود، اما پیاده‌سازی باید به‌گونه‌ای باشد که منطق بازی برای هر اندازه انتخاب‌شده به‌درستی عمل کند.

Battle Hangman (۰.۲۵)

در این بخش شما نسخه‌ای رقابتی از بازی (Hangman) را پیاده‌سازی می‌کنید که به‌صورت حالت نبرد انجام می‌شود. در این حالت، دو بازیکن به‌طور هم‌زمان در بازی شرکت می‌کنند و هر کدام یک کلمه مخفی برای حریف خود انتخاب می‌کند. بازیکنان به‌صورت نوبتی حروف را حدس می‌زنند تا کلمه مخفی طرف مقابل را کشف کنند.

قانون اصلی بازی به این صورت است که هر بار بازیکن یک حرف را حدس می‌زند. اگر حرف حدس‌زده‌شده در کلمه حریف وجود داشته باشد، تمام موقعیت‌های آن حرف نمایش داده می‌شود. در غیر این صورت، یک بخش از شکل چوبه‌دار مربوط به همان بازیکن تکمیل می‌شود. برخلاف نسخه تک‌نفره، در این حالت هر بازیکن مسئول اشتباهات خودش است.

هدف بازی این است که یک بازیکن زودتر از حریف بتواند کلمه او را به‌طور کامل حدس بزند، یا اینکه حریف با تکمیل کامل شکل چوبه‌دار خود، بازی را ببازد.

نمونه State اول و پایانی یک راند از بازی

```

+---+
|   |
|   |
|   |
|   |
|   |
=====

```

```

+---+
|   |
|   0
|  /|\
|  / \
|
=====

```

وظیفہ شما

شما باید نوبت بازیکنان را به‌طور کامل مدیریت کنید و پس از هر حدس، نوبت را به بازیکن دیگر منتقل نمایید. وضعیت کلمه، حروف استفاده‌شده و وضعیت چوبه‌دار هر بازیکن باید به‌صورت مستقل نگهداری شود.

نمرهٔ مازاد

در صورت پیاده‌سازی کامل نسخهٔ دوم، امکانات زیر به‌عنوان نمرهٔ مازاد در نظر گرفته می‌شوند:

- افزودن حریف Bot
- امکان انتخاب تصادفی کلمه از یک لیست کلمات

(. . 2 .) TicTacToe Limited

در این بازی شما نسخه‌ای تغییر یافته از بازی کلاسیک دوز یا Tic-Tac-Toe را پیاده‌سازی می‌کنید. صفحه بازی یک جدول 3×3 است و دو بازیکن به صورت نوبتی مهره‌های خود را روی صفحه قرار می‌دهند. هدف بازی، مانند نسخه استاندارد، تشکیل یک خط سه تایی افقی، عمودی یا قطری است.

تغییر اصلی این نسخه در محدودیت تعداد مهره‌های هر بازیکن است. هر بازیکن تنها مجاز است حداکثر سه مهره به‌طور هم‌زمان روی صفحه داشته باشد. زمانی که بازیکن بخواهد مهره چهارم خود را قرار دهد، قدیمی‌ترین مهره او به‌صورت خودکار از صفحه حذف می‌شود. این حذف بر اساس ترتیب زمانی قرارگیری مهره‌ها انجام می‌شود و بازیکن کنترل روی انتخاب مهره حذف‌شده ندارد. این قانون باعث می‌شود بازی همواره پویا باقی بماند و وضعیت صفحه به‌طور مداوم تغییر کند.

وظیفه شما

در پیاده‌سازی این بازی، شما باید ترتیب قرارگیری مهره‌های هر بازیکن را به‌درستی ثبت کنید و هنگام قرار دادن مهره جدید، منطق حذف خودکار مهره قدیمی را اعمال نمایید. همچنین باید تشخیص برد را پس از هر حرکت بررسی کنید و از انجام حرکت‌های غیرمجاز جلوگیری نمایید. مدیریت نوبت‌ها، نمایش صفحه بازی و اعلام نتیجه نهایی از وظایف اصلی شما در این بخش است.

شکل یکی از State های بازی

3	2	1	
0		.	X 1
----	+	----	+
.		X	. 2
----	+	----	+
0		.	. 3

سنگ کاغذ قیچی ۵ حرکتی (۰.۱۵)

در این بخش شما نسخه‌ای گسترش‌یافته از بازی سنگ-کاغذ-قیچی را پیاده‌سازی می‌کنید. به‌جای سه حرکت کلاسیک، بازی شامل پنج حرکت مختلف است که روابط برد و باخت مشخصی با یکدیگر دارند.

هر دور بازی، بازیکن یک حرکت را انتخاب می‌کند و نتیجه آن بر اساس جدول روابط از پیش تعریف‌شده مشخص می‌شود. بازی می‌تواند به‌صورت تک‌دور یا چنددور انجام شود و نتیجه هر دور باید به‌وضوح اعلام گردد.

روابط بین موجودیت‌های بازی

Lizard ,Scissors >	Rock
Spock ,Rock >	Paper

Lizard ,Paper > Scissors
Spock ,Paper > Lizard
Scissors ,Rock > Spock

وظیفه شما

در پیاده‌سازی این بازی، شما باید منطق تصمیم‌گیری برنده و بازنده را به صورت کامل پیاده‌سازی کنید. تمام ترکیب‌های ممکن بین حرکات باید پوشش داده شوند و نتیجه هر ترکیب به درستی تعیین گردد. دریافت ورودی از کاربر، تولید انتخاب حریف و نمایش نتیجه هر دور از وظایف اصلی شماست. بازی باید هم در حالت دونفره و هم در حالت تک‌نفره در برابر Bot تصادفی قابل اجرا باشد.

نمره مازاد

در صورت پیاده‌سازی Bot با رفتار غیرتصادفی و تصمیم‌گیری هوشمندتر از انتخاب کاملاً رندوم، نمره مازاد به این بخش تعلق می‌گیرد.

حدس عدد پیشرفته (۰.۱۰)

در این بازی، برنامه باید در ابتدای اجرا یک عدد مخفی را به صورت تصادفی در یک بازه مشخص انتخاب کند. بازیکن با وارد کردن حدس‌های متوالی تلاش می‌کند این عدد مخفی را پیدا کند. بازی تا زمانی ادامه پیدا می‌کند که بازیکن عدد درست را حدس بزند یا تعداد تلاش‌های مجاز (در صورت تعریف محدودیت) به پایان برسد.

در نسخه پایه بازی حدس عدد، تنها راهنمایی ارائه شده به بازیکن این است که حدس وارد شده از عدد مخفی «بزرگ‌تر» یا «کوچک‌تر» است. در این پروژه، شما باید نسخه پیشرفته‌تری از این بازی را پیاده‌سازی کنید که در آن راهنمایی‌های متنوع و معنادار به بازیکن ارائه می‌شود و بازیکن بتواند با استفاده از این اطلاعات تصمیم‌گیری بهتری انجام دهد.

راهنمایی‌های پیشرفته می‌توانند شامل موارد مختلفی باشند. برای مثال، برنامه می‌تواند مشخص کند که عدد مخفی زوج است یا فرد، یا این که حدس بازیکن از عدد مخفی «خیلی نزدیک»، «نسبتاً نزدیک» یا «دور» است (مفهوم گرم و سرد). همچنین می‌توانید بازه‌های تقریبی ارائه دهید؛ برای نمونه اعلام کنید که عدد مخفی در بازه‌ای مانند ۲۰ تا ۳۰ قرار دارد. نوع و دقت این راهنمایی‌ها می‌تواند بسته به سطح دشواری بازی متفاوت باشد.

برای افزایش عمق بازی، باید حداقل سه سطح دشواری مختلف در نظر گرفته شود. در سطح آسان، راهنمایی‌ها دقیق‌تر و کمک‌کننده‌تر هستند و ممکن است پس از هر حدس نادرست، بازه

جست‌وجو کوچک‌تر شود. در سطح سخت، راهنمایی‌ها مبهم‌تر هستند و اطلاعات کمتری در اختیار بازیکن قرار می‌گیرد. انتخاب سطح دشواری باید در ابتدای بازی انجام شود.

وظیفه شما

در پیاده‌سازی این بازی، شما باید منطق تولید عدد مخفی را به صورت تصادفی و در بازه‌ای مشخص پیاده‌سازی کنید. همچنین باید ورودی‌های کاربر را دریافت کرده و از معتبر بودن آن‌ها اطمینان حاصل کنید؛ به عنوان مثال، ورودی‌های غیر عددی یا خارج از بازه نباید پذیرفته شوند.

پس از هر حدس، برنامه باید راهنمایی‌های مناسب را بر اساس سطح دشواری انتخاب‌شده نمایش دهد. لازم است این راهنمایی‌ها با یکدیگر سازگار باشند و بازیکن را دچار تناقض نکنند. شما باید تعداد تلاش‌های انجام‌شده را شمارش کنید و در صورت حدس صحیح، بازی را با اعلام موفقیت بازیکن به پایان برسانید.

در صورتی که برای بازی محدودیت تعداد تلاش تعریف کرده‌اید، باید پایان بازی در حالت شکست نیز به درستی مدیریت شود. در پایان هر دور بازی، نتیجه باید به طور واضح به بازیکن اعلام شود و امکان شروع مجدد یا خروج از بازی فراهم گردد.

در نهایت...

در صورتی که در حین انجام پروژه با ابهام، سؤال یا مشکل فنی مواجه شدید، سؤالات خود را از اعضای تیم حل‌تمرین درس بپرسید. توجه داشته باشید که مسئولیت نهایی طراحی، پیاده‌سازی و صحت کد بر عهده شما خواهد بود.

با آرزوی موفقیت.